

UserManagementApp Project Documentation

This document describes the current implementation of the UserManagementApp solution, including API, desktop client, data model, authentication, validation, testing, logging, and planned UI/reporting work.

Section 1 - API

1.1 Models

The core domain contains User and SupportRequest models. User stores identity fields, role, password hash, creation/update timestamps, and a collection of related support requests. SupportRequest links to User through UserId and tracks subject, message, status, and creation time.

1.2 Database Configuration

Entity Framework Core with SQLite is configured through AppDbContext. The Users table has required UserName, Email, PasswordHash, Role, FirstName, and CreatedAt fields, max lengths for key text fields, string enum conversion, and unique indexes for UserName and Email. SupportRequests are configured with required subject/message/status fields and cascade delete from User.

1.3 Authentication

AuthController exposes POST /api/auth/login. AuthService loads a user by username, verifies the submitted password through IPasswordHasher, and returns LoginResponse with user id, username, email, and role when successful. PasswordHasher wraps Microsoft.AspNetCore.Identity PasswordHasher so newly created users store hashed passwords instead of plaintext.

1.4 Validation

Database-level constraints exist in AppDbContext, but request DTOs currently do not use DataAnnotations. This means invalid API input may reach the service layer and fail as a database exception instead of returning a clean 400 response.

1.5 Services

UserService implements user listing, lookup, creation, update, and deletion. It maps User entities into UserResponse DTOs and hashes passwords during user creation. PermissionService checks whether a supplied user id belongs to an admin user.

1.6 Controllers

UsersController exposes CRUD endpoints under /api/users. Update and delete operations check admin permissions using the X-User-Id header, while create/list/get endpoints currently do not enforce equivalent authorization. DatabaseController exposes /api/database/status for health-style database connectivity checks.

1.7 Swagger/OpenAPI

Swagger/OpenAPI services are registered with AddEndpointsApiExplorer and AddSwaggerGen. Swagger UI is enabled only in Development.

Section 2 - Desktop Application

2.1 Console Client

The desktop project is currently a console client. It supports login, logout, listing users, and admin-only menu options for creating, updating, and deleting users. It stores the current LoginResponse in memory to decide which menu actions to show.

2.2 API Integration

AuthApiClient and UserApiClient wrap HttpClient calls to the API. UserApiClient sends X-User-Id for requests that need permission context and returns typed DTOs for successful responses.

2.3 Configuration Handling

The console app loads appsettings.json from AppContext.BaseDirectory. ApiSettings currently contains the API BaseUrl, defaulting to http://localhost:5236.

Section 3 - Testing

3.1 Unit Tests

Unit tests cover UserService and AuthService behavior with EF Core InMemory databases. They verify user CRUD behavior, login success/failure, and password hashing expectations.

3.2 API Tests

API tests call a running API instance through HttpClient. They cover authentication, users CRUD paths, forbidden/not-found cases, and database status. Tests use typed API clients and ApiTestResponse wrappers so status code, success flag, raw body, and typed body can be asserted together.

3.3 Test Structure

The test project is organized into ApiTests, IntegrationTests, UnitTests, and Core helper folders. BaseApiTest centralizes HttpClient setup, test data helpers, API clients, and cleanup registration.

3.4 Logging

TestStepLogger adds readable step-level output and failure messages through TestContext.Progress. TestLog adds request/response logging with optional response body output, which is useful when API tests fail.

3.5 Test Data Handling

TestDataHelper creates or updates admin and regular test users directly in the SQLite test database, hashing passwords when needed. Temporary users are registered for cleanup and are deleted after API tests through an admin cleanup client.

Planned Features / To Be Implemented

The next stage of the project is expected to introduce a WPF-based UI and richer testing and reporting around the main user flows.

- TASK-18: Build Login UI - create a WPF login screen connected to the existing auth endpoint.
- TASK-19: Build Admin Dashboard UI - provide admin workflows for user management and overview data.
- TASK-20: Build User Dashboard UI - provide a regular user landing area with role-appropriate actions.
- TASK-21: Build Support Request UI - let users create and track support requests, and let admins manage them.
- TASK-22: Write UI tests for main flows - automate login, dashboards, user management, and support request scenarios.
- Advanced logging - add structured app/test logs with correlation between actions and API requests.
- Allure reporting - publish test steps, attachments, request/response data, and failure details.
- UI automation - likely using WPF-friendly automation tooling for repeatable end-to-end coverage.

Current Review Notes

The current implementation has a clean solution split into Core, Data, API, Desktop, and Tests. The password hasher abstraction, EF model configuration, and test helper structure are strong foundations. Main remaining work is to replace spoofable header-based authorization with real authentication, add DTO validation, make API tests self-hosted or environment-isolated, and finish support request endpoints.